

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Diseño de Lenguajes de Programación

Semestre I – 2026



Proyecto 2

Integrantes:

Nájera Marakovits, Ingrid Nina Alessandra - 231088

Ramírez Velásquez, Diego Alejandro – 23601

Eliazar José Pablo Canastuj Matías - 23384

Enlace al repositorio: https://github.com/Ninaswiftie09/Proyecto2_Compis

Introducción

El proyecto consiste en la implementación de un generador de analizadores sintácticos. El sistema recibe como entrada un archivo escrito en YAPar, construye un analizador sintáctico SLR y lo integra con un analizador léxico generado a partir de una especificación YALex. El resultado final permite procesar un archivo de texto plano, convertirlo en un flujo de tokens y parsear ese flujo usando una tabla SLR. El programa también informa errores léxicos y sintácticos, genera archivos Python independientes para el lexer y el parser, y exporta representaciones del AFD léxico y del autómata LR(0).

Desde el punto de vista de compiladores, el proyecto cubre dos fases principales: el análisis léxico, donde el texto de entrada se transforma en tokens, y el análisis sintáctico SLR, donde la secuencia de tokens se valida contra una gramática libre de contexto.

Objetivos

- Aplicar la teoría de análisis sintáctico en la construcción de una herramienta generadora de parsers.
- Leer especificaciones sintácticas escritas en YAPar y convertirlas en una estructura de gramática.
- Calcular los conjuntos Primero y Siguiente necesarios para construir la tabla SLR.
- Construir el autómata LR(0) a partir de la gramática aumentada.
- Generar la tabla ACTION/GOTO para realizar el análisis sintáctico.
- Lograr la interacción entre el analizador léxico generado y el analizador sintáctico generado.
- Reportar de forma clara los errores léxicos y sintácticos encontrados durante el análisis.

Arquitectura por módulos

Requerimiento	Archivo
Leer archivo YALex	yalex_parser.py lee reglas TOKEN, IGNORE, let y reglas estilo YALex.
Leer archivo YAPar	yalp_parser.py separa tokens y producciones usando %% y construye un objeto Grammar.
No usar re para lexer	regex_engine.py implementa tokenización de regex, postfix y Thompson manualmente.
Usar autómatas finitos	regex_engine.py y lexer_builder.py trabajan con

	AFN, AFD y simulación de AFD.
Construir autómata LR(0)	slr_builder.py implementa closure, goto y colección LR(0).
Construir tabla SLR	slr_builder.py calcula ACTION/GOTO usando FOLLOW para reducciones.
Parser interactúa con lexer	pipeline.py coordina tokenización y parseo del flujo de tokens.
Errores léxicos y sintácticos	El lexer reporta errores de reconocimiento y parse_tokens reporta tokens esperados.
Lexer y parser independientes	codegen.py genera generated_lexer.py y generated_parser.py autocontenidos.
Interfaz gráfica	ui_app.py implementa la interfaz con tkinter.
Tres niveles de prueba	tests_data/low, tests_data/medium y tests_data/high contienen archivos de prueba.

Diseño del software

El software se diseñó por capas. Cada capa tiene una responsabilidad específica: lectura de especificaciones, construcción de estructuras teóricas, generación de código, interfaz gráfica y ejecución de pruebas. Esta separación permite modificar o depurar cada componente sin afectar directamente a los demás.

Módulo	Responsabilidad	Importancia
main.py	Punto de entrada principal.	Permite iniciar el programa.
ui_app.py	Interfaz gráfica con tkinter.	Cumple la restricción de una UI amigable.
models.py	Define estructuras como Grammar, LexRule, YalSpec e Item.	Evita duplicación de datos entre módulos.
yalex_parser.py	Lee la especificación léxica .yal.	Convierte texto YALex en reglas de tokens.
regex_engine.py	Implementa motor propio de expresiones regulares.	Evita usar re y construye AFN por Thompson.
lexer_builder.py	Construye AFD y tokeniza	Produce el flujo de tokens

	entrada.	para el parser.
yalp_parser.py	Lee la especificación sintáctica .yalp.	Construye la gramática libre de contexto.
grammar_tools.py	Calcula Primero y Siguiente.	Necesario para colocar reducciones SLR.
slr_builder.py	Construye LR(0), tabla SLR y ejecuta parseo.	Núcleo del analizador sintáctico.
codegen.py	Genera código Python independiente.	Cumple la independencia del lexer y parser generados.
pipeline.py	Orquesta el proceso completo.	Conecta generación, exportación y ejecución.

Flujo de funcionamiento del generador

1. El usuario selecciona un archivo .yal, un archivo .yalp y un archivo de entrada.
2. El sistema lee el .yal y obtiene las reglas léxicas.
3. Las expresiones regulares se convierten a postfix y luego a AFN mediante Thompson.
4. El AFN se convierte en AFD para reconocer tokens de forma determinista.
5. El sistema lee el .yalp, declara terminales y extrae producciones.
6. Se construye la gramática aumentada.
7. Se calculan los conjuntos Primero y Siguiente.
8. Se construye la colección canónica de ítems LR(0) mediante Cerradura e Ir_A.
9. Se genera la tabla SLR ACTION/GOTO.
10. Se generan archivos Python independientes para el lexer y parser.
11. Se exportan representaciones textuales o visuales del AFD y LR(0).

Funcionamiento del analizador generado

Después de generar los analizadores, el flujo de ejecución sobre una entrada de texto plano es el siguiente:

1. El lexer generado lee caracteres del archivo de entrada.
2. El lexer reconoce el token más largo posible usando el AFD.
3. Si el token pertenece a IGNORE, se omite y se continúa con el siguiente lexema.
4. Si el token es válido, se entrega al parser como par (TOKEN, lexema).
5. El parser consulta ACTION usando el estado actual y el token recibido.
6. Si ACTION indica shift, avanza en la entrada y apila el nuevo estado.
7. Si ACTION indica reduce, aplica una producción y consulta GOTO.
8. Si ACTION indica accept, la cadena es aceptada por la gramática.
9. Si no existe acción, se reporta error sintáctico con los tokens esperados.

Instrucciones de ejecución del programa

Requisitos previos

Antes de ejecutar el programa, se debe verificar que Python 3 esté instalado en el sistema. Para comprobarlo, se puede abrir una terminal y ejecutar el siguiente comando: `python --version`

Ejecucion

Para ejecutar el programa, primero se debe ingresar a la carpeta principal del proyecto: `cd Proyecto2_Compis` Desde esta ubicación se deben ejecutar los comandos principales del sistema.

La forma principal de ejecutar el programa es utilizando la interfaz gráfica incluida en el proyecto. Para ello, se debe ejecutar el archivo principal: `python src/main.py`

Al ejecutar este comando, se abrirá una interfaz gráfica desarrollada con tkinter. Desde esta interfaz, el usuario puede seleccionar los archivos necesarios para el análisis: el archivo léxico `.yal`, el archivo sintáctico `.yalp` y el archivo de entrada `.txt`.

La interfaz permite generar los analizadores léxico y sintáctico, visualizar información relacionada con los autómatas y ejecutar el análisis completo sobre el archivo de entrada seleccionado.

Después de ejecutar el programa, se generan archivos de salida en la carpeta indicada mediante el parámetro `--out`. Por ejemplo, si se utiliza:

`--out outputs/medium_run`

los resultados se almacenarán dentro de la carpeta:

`outputs/medium_run`

Además, el sistema genera analizadores independientes en Python, tales como:

`generated_lexer.py`
`generated_parser.py`

Estos archivos corresponden al analizador léxico y al analizador sintáctico generados a partir de las especificaciones proporcionadas.

Casos de prueba

Se realizaron tres grupos de prueba: baja, media y alta complejidad. En cada grupo se incluyó una especificación léxica, una especificación sintáctica y un archivo de entrada.

Nivel	Archivos esperados	Qué valida
Baja	lexer_low.yal, parser_low.yalp, input_low.txt	Tokens simples, gramática pequeña y aceptación básica.
Media	lexer_medium.yal, parser_medium.yalp, input_medium.txt	Expresiones con operadores, precedencia básica y más producciones.
Alta	lexer_high.yal, parser_high.yalp, input_high.txt	Gramática más compleja, anidamientos, varios tokens y posibles errores.

Ejemplos de uso:

Generador de analizadores SLR - YALex + YAPar

Generador de analizadores SLR

Carga YALex, YAPar y una entrada. La herramienta genera lexer/parser independientes, AFD, LR(0), tabla SLR y traza de parsing.

Flujo de trabajo

Sigue estos 4 pasos para generar y probar.

1. Especificación léxica (.yal)

Archivo con reglas TOKEN / IGNORE

Buscar

2. Especificación sintáctica (.yalp)

Archivo YAPar con %token, IGNORE, %% y producciones

Buscar

3. Entrada de prueba (.txt)

Texto plano que será tokenizado y parseado

Buscar

4. Carpeta de salida

Aquí se guardan los analizadores y diagramas

Buscar

Ejemplos rápidos

Low

Medium

High

Generar y analizar

Limpiar resultados

Resultados del análisis

ACEPTADO

Tokens

Traza SLR

Errores

Archivos generados

AFD

LR(0)

Tabla SLR

Token	Lexema
NUM	'1'
PLUS	'+'
NUM	'2'
PLUS	'+'
NUM	'3'
\$	'\$'

Generador de analizadores SLR - YALex + YAPar

Generador de analizadores SLR

Carga YALex, YAPar y una entrada. La herramienta genera lexer/parser independientes, AFD, LR(0), tabla SLR y traza de parsing.

Flujo de trabajo

Sigue estos 4 pasos para generar y probar.

1. Especificación léxica (.yal)

Archivo con reglas TOKEN / IGNORE

Buscar

2. Especificación sintáctica (.yalp)

Archivo YAPar con %token, IGNORE, %% y producciones

Buscar

3. Entrada de prueba (.txt)

Texto plano que será tokenizado y parseado

Buscar

4. Carpeta de salida

Aquí se guardan los analizadores y diagramas

Buscar

Ejemplos rápidos

Low

Medium

High

Generar y analizar

Limpiar resultados

Resultados del análisis

ACEPTADO

Tokens

Traza SLR

Errores

Archivos generados

AFD

LR(0)

Tabla SLR

NUM

expr -> • expr PLUS term

expr -> • term

expr' -> • expr

term -> • NUM

expr

I2

expr -> expr • PLUS term

expr' -> expr •

term

I3

expr -> term •

I4

expr -> expr PLUS • term

term -> • NUM

4

expr -> • expr PLUS term

expr -> • term

expr' -> • expr

term -> • NUM

goto(NUM) = I1

goto(expr) = I2

goto(term) = I3

I1

term -> NUM •

Resultados del análisis						ACEPTADO
Tokens	Traza SLR	Errores	Archivos generados	AFD	LR(0)	Tabla SLR
ACTION state \$ NUM PLUS 0 s1 1 r(term->NUM) r(term->NUM) 2 acc s4 3 r(expr->term) r(expr->term) 4 s1 5 r(expr->expr PLUS term) r(expr->expr PLUS term) GOTO state expr term 0 2 3 1 2 3 4 5 5						

Documentación y explicación del código

yalex_parser.py

Este módulo interpreta el archivo YALex. Su objetivo no es reconocer la entrada final, sino leer la especificación del lexer. El resultado es una lista de reglas léxicas con nombre de token, expresión regular, indicador de ignore y línea fuente.

- `_strip_comments` elimina comentarios de bloque, comentarios con `//` y comentarios con `#`.
- `_split_once_keyword` detecta líneas que empiezan con palabras clave como `TOKEN`, `IGNORE` o `let`.
- `_parse_action_token` extrae el token de acciones como `{ return NUM }`.
- `_extract_rule_line` lee reglas tipo `| regex { return TOKEN }`.
- `_replace_definitions` reemplaza nombres definidos con `let` por sus expresiones regulares.
- `parse_yalex` coordina toda la lectura, valida reglas y devuelve un `YalSpec`.

regex_engine.py

Este módulo implementa el motor de expresiones regulares del proyecto. El archivo transforma expresiones regulares en AFN.

Función	Qué hace
<code>tokenize_regex</code>	Convierte un patrón en tokens internos: literales, operadores, clases y paréntesis.
<code>add_concat</code>	Agrega el operador explícito de concatenación.
<code>to_postfix</code>	Convierte la expresión a notación postfix

	usando precedencia.
thompson	Construye un AFN con transiciones epsilon.
epsilon_closure	Calcula estados alcanzables por transiciones epsilon.

yalp_parser.py

Este módulo lee la especificación YAPar. El archivo .yalp se divide en sección de tokens y sección de producciones usando %%. La función principal parse_yalp devuelve una estructura Grammar con terminales, no terminales, símbolo inicial, producciones e ignores.

- Valida que exista el separador %%.
- Declara terminales usando %token o token.
- Permite tokens IGNORE.
- Lee producciones separadas por punto y coma.
- Lee alternativas separadas por barra vertical.
- Toma como símbolo inicial el lado izquierdo de la primera producción.
- Valida que los tokens usados estén declarados y que los no terminales tengan producción propia.

grammar_tools.py

Este archivo implementa los conjuntos Primero y Siguiente de la gramática. Estos conjuntos son los que se usan en SLR para saber en qué columnas de ACTION colocar una reducción.

Función	Explicación
first_of_sequence(sequence, first, terminals)	Calcula Primero de una secuencia de símbolos, considerando epsilon.
first_sets(grammar)	Calcula Primero para cada no terminal hasta que ya no haya cambios.
follow_sets(grammar, first)	Calcula Siguiente para cada no terminal; agrega \$ al símbolo inicial.

slr_builder.py

Este es el módulo central del parser. Implementa Cerradura, Ir_A/GOTO, gramática aumentada, construcción de la colección LR(0), generación de la tabla SLR y ejecución del algoritmo de análisis LR.

Función	Responsabilidad
closure	Calcula Cerradura(I). Agrega producciones de no terminales que aparecen después del punto.

goto	Implementa Ir_A(I, X). Avanza el punto sobre X y aplica closure.
_augment_grammar	Agrega una nueva producción inicial S' -> S.
build_slr	Construye estados LR(0), transiciones, ACTION, GOTO y conflictos.
parse_tokens	Ejecuta el parser SLR sobre el flujo de tokens.
lr0_to_text	Exporta estados LR(0) a texto.
lr0_to_dot	Genera representación DOT del autómata LR(0).
table_to_text	Exporta las tablas ACTION y GOTO en texto.

Explicación detallada del parser SLR

Gramática aumentada

Antes de construir el autómata LR(0), el programa transforma la gramática original en una gramática aumentada. Para ello, agrega un nuevo símbolo inicial y una producción adicional que apunta al símbolo inicial original. Esta modificación permite identificar de forma clara cuándo el análisis sintáctico ha terminado correctamente y debe ejecutarse la acción de aceptación.

Ítems LR(0)

Un ítem LR(0) representa una producción de la gramática junto con una marca de posición, usualmente representada por un punto. Este punto indica qué parte de la producción ya ha sido reconocida y qué parte falta por analizar. Los ítems permiten describir los estados internos del parser durante el proceso de reconocimiento.

Cerradura(I)

La operación de cerradura permite completar un conjunto de ítems LR(0). Cuando el punto se encuentra antes de un no terminal, se agregan al conjunto las producciones de ese no terminal con el punto al inicio. Este proceso se repite hasta que ya no sea posible agregar nuevos ítems.

Ir_A(I, X) / GOTO

La operación Ir_A, también conocida como GOTO, permite avanzar de un estado a otro dentro del autómata LR(0). Esta operación mueve el punto sobre un símbolo determinado y luego aplica cerradura al conjunto resultante. De esta manera se generan las transiciones entre estados del autómata.

Construcción del autómata LR(0)

La construcción del autómata LR(0) inicia con el estado inicial obtenido a partir de la gramática aumentada. Luego, para cada estado y para cada símbolo de la gramática, se calcula la operación GOTO. Si el resultado produce un conjunto de ítems nuevo, este se agrega como un nuevo estado. El proceso continúa hasta que no se generen más estados nuevos.

Construcción de la tabla SLR

La tabla SLR se construye utilizando el autómata LR(0) y los conjuntos Primero y Siguiente de la gramática. Esta tabla se divide en dos partes principales: ACTION y GOTO. La sección ACTION define las acciones que el parser debe ejecutar cuando se encuentra en un estado determinado y recibe un token específico de entrada. Las acciones posibles incluyen desplazamiento, reducción, aceptación o error. La sección GOTO define las transiciones entre estados cuando se reconoce un no terminal durante el proceso de reducción.

Algoritmo de análisis LR

El analizador sintáctico mantiene una pila de estados que representa el progreso del análisis. En cada iteración, el parser consulta la tabla ACTION utilizando el estado actual y el token de entrada. Dependiendo de la acción obtenida, el parser realiza una operación específica sobre la pila y sobre la entrada restante.

Acción	Comportamiento
shift	El parser consume el token actual y avanza hacia un nuevo estado del autómata.
reduce	El parser aplica una producción de la gramática, reduciendo una secuencia reconocida a un no terminal.
accept	El análisis finaliza correctamente y la entrada pertenece al lenguaje definido por la gramática.
error	No existe una acción válida para el estado y token actuales, por lo que se reporta un error sintáctico.

Manejo de errores

Errores léxicos

Un error léxico ocurre cuando el analizador léxico no puede reconocer ningún token válido desde la posición actual del archivo de entrada. En ese caso se reporta el fragmento problemático y se evita entregar un flujo inválido al parser.

Errores sintácticos

Un error sintáctico ocurre cuando no existe una acción ACTION[estado, token]. El método parse_tokens devuelve un mensaje con el token leído, su lexema, el estado actual y los tokens esperados.

Conflictos SLR

Durante la construcción de ACTION puede ocurrir que una celda reciba dos acciones diferentes. Esto se registra como conflicto SLR. El conflicto puede ser shift/reduce o reduce/reduce, y normalmente indica que la gramática no es SLR.

Conclusiones

El proyecto implementa un generador de analizadores sintácticos SLR que cumple con los objetivos principales del enunciado. La arquitectura separa claramente el análisis léxico, el análisis sintáctico, la generación de código y la interfaz gráfica.

La parte sintáctica sigue el procedimiento visto en clase: lectura de gramática, cálculo de Primero y Siguiente, gramática aumentada, Cerradura, Ir_A/GOTO, construcción del autómata LR(0), tabla ACTION/GOTO y ejecución con shift, reduce y accept.

La parte léxica cumple la restricción de no usar librerías de expresiones regulares, ya que implementa un motor propio basado en expresiones regulares, notación postfix, construcción de Thompson y autómatas finitos.